Title of the Project: Rising Waters
ML Flood Prediction System

Abstract:

Rising Waters is a Machine Learning-based web application designed to predict flood risk using weather and rainfall parameters. The system analyzes user-provided environmental data such as annual rainfall, seasonal rainfall, temperature, humidity, and cloud visibility using a trained XGBoost model. The application is developed using Python and Flask, providing users with a simple and interactive interface for instant flood prediction. By integrating machine learning with a web application, the system supports early flood risk assessment and helps improve disaster preparedness.

Problem Statement:

Floods are among the most destructive natural disasters, causing severe damage to lives, infrastructure, agriculture, and the environment. Traditional flood prediction methods often require manual analysis and may not provide timely or accurate warnings. There is a need for an intelligent system that can analyze weather and rainfall data quickly and accurately to predict flood occurrence, enabling authorities and communities to take preventive measures and reduce potential losses.

Features:

- Predicts flood occurrence using Machine Learning.
- User-friendly web interface for entering weather and rainfall parameters.
- Data preprocessing and feature scaling before prediction.
- XGBoost-based prediction model for improved accuracy.
- Displays Flood Chance and No Flood Chance result pages.
- Shows flood probability and safe condition probability.
- Responsive frontend developed using HTML, CSS, and JavaScript.
- Fast prediction through Flask backend.
- Cloud deployment using Render.
- Modular project structure for easy maintenance and scalability.

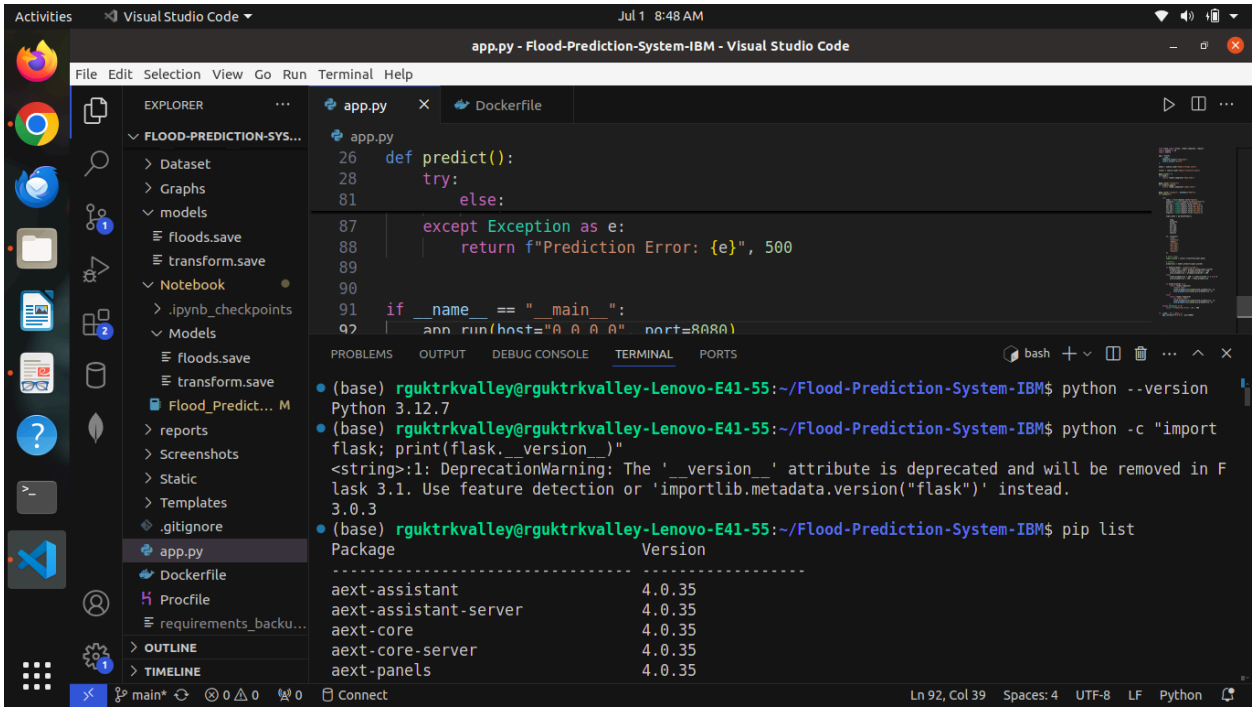
Tech Stack:

Type of Tech Stack	Technologies Used
Frontend	HTML,CSS, JavaScript
Backend	Python, Flask

Machine Learning	XGBoost, Scikit-learn, Joblib
Data Preprocessing	Pandas, NumPy
Data Visualization	Matplotlib, Seaborn
Development Tools	Jupyter Notebook, Visual Studio Code, Git, GitHub
Deployment	Render, Gunicorn

Installation Steps:

1. Install Python 3.8 or later.
2. Clone or download the project repository.
3. Open the project folder in Visual Studio Code.
4. Install all required dependencies using:
   pip install -r requirements.txt
5. Ensure the trained model files (`floods.save` and `transform.save`) are available inside the `models` folder.
6. Run the Flask application:
   python app.py



```
(base) rguktrkvalley@rguktrkvalley-Lenovo-E41-55:~/Flood-Prediction-System-IBM$ pip install
 -r requirements.txt
Requirement already satisfied: Flask in /home/rguktrkvalley/anaconda3/lib/python3.12/site-p
ackages (from -r requirements.txt (line 1)) (3.0.3)
Requirement already satisfied: gunicorn in /home/rguktrkvalley/anaconda3/lib/python3.12/sit
e-packages (from -r requirements.txt (line 2)) (26.0.0)
Requirement already satisfied: numpy in /home/rguktrkvalley/anaconda3/lib/python3.12/site-p
ackages (from -r requirements.txt (line 3)) (1.26.4)
Requirement already satisfied: pandas in /home/rguktrkvalley/anaconda3/lib/python3.12/site-
packages (from -r requirements.txt (line 4)) (2.2.2)
Requirement already satisfied: scikit-learn in /home/rguktrkvalley/anaconda3/lib/python3.12
/site-packages (from -r requirements.txt (line 5)) (1.5.1)
Requirement already satisfied: joblib in /home/rguktrkvalley/anaconda3/lib/python3.12/site-
packages (from -r requirements.txt (line 6)) (1.4.2)
```

The project directory was organized as:

Flood-Prediction-System-IBM/

├── app.py

├── Dataset

│ └── flood dataset.xlsx

├── Dockerfile

├── Graphs

├── models

│ ├── floods.save

│ └── transform.save

├── Notebook

│ ├── Flood_Prediction.ipynb

│ └── Models

│ ├── floods.save

│ └── transform.save

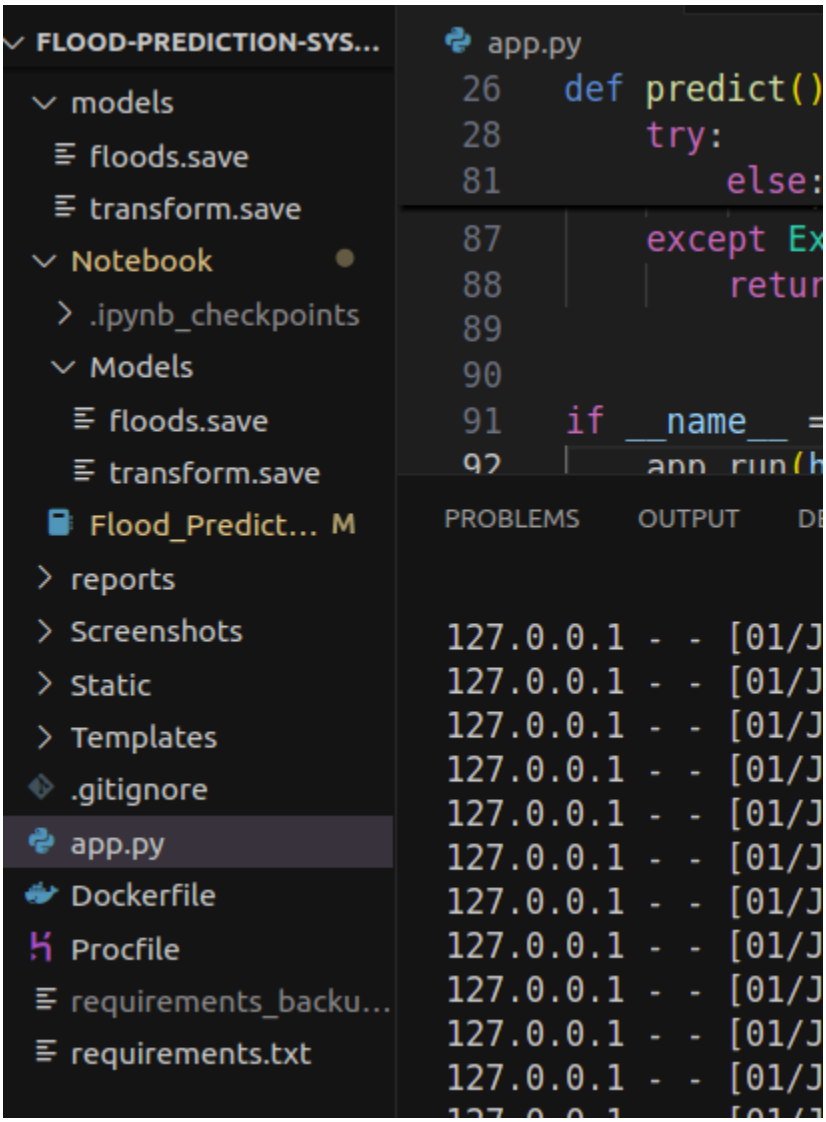
├── Procfile

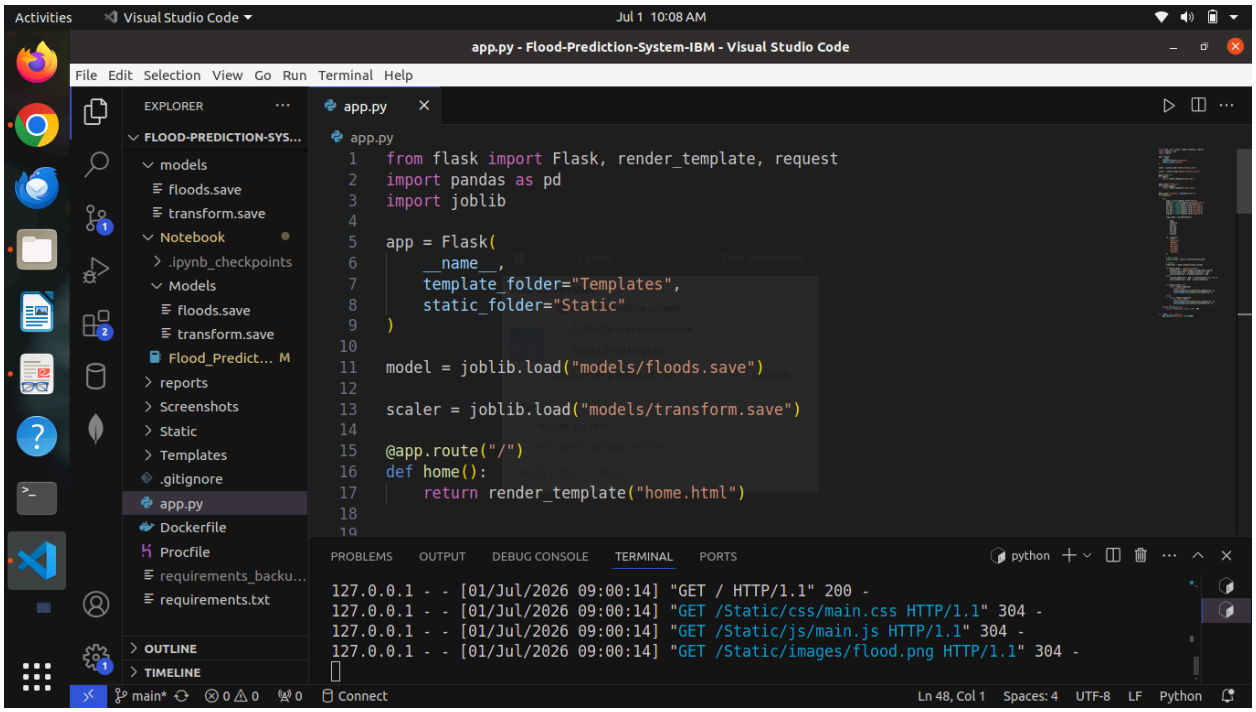
├── __pycache__

```
│   └── app.cpython-312.pyc
├── README.md
├── reports
│   └── Flood_Report.pdf
├── requirements_backup.txt
├── requirements.txt
├── Screenshots
│   ├── app.png
│   ├── flastpython_install.png
│   ├── flood 1.png
│   ├── flood1.png
│   ├── flood .png
│   ├── git.png
│   ├── home.png
│   ├── livedeployeswebsite.png
│   ├── noflood1.png
│   ├── noflood.png
│   ├── predictioninputs.png
│   ├── render.png
│   ├── requirementinstall.png
│   ├── requirements.png
│   └── runapppy.png
├── Static
│   ├── css
```

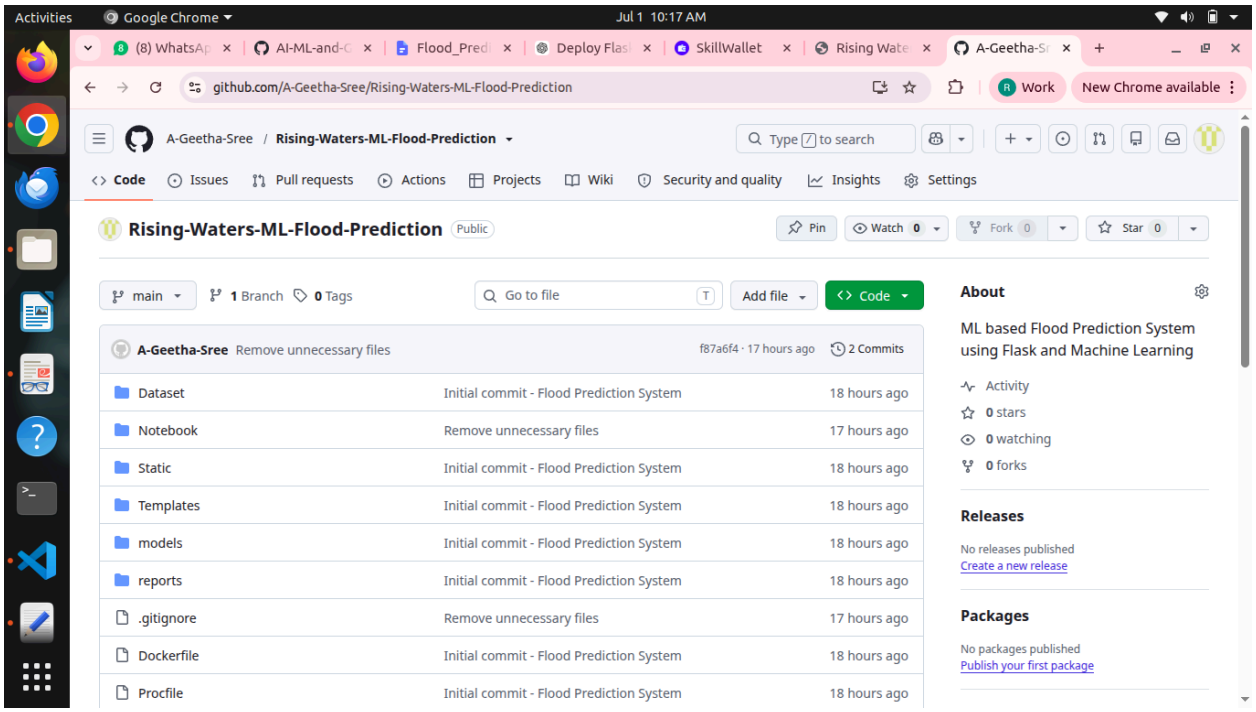
```
│   │   ├── chance.css
│   │   ├── main.css
│   │   ├── no_chance.css
│   │   └── predict.css
│   ├── images
│   │   ├── flood.png
│   │   ├── logo.png
│   │   ├── safe.png
│   │   └── warning.png
│   └── js
│       ├── chance.js
│       ├── main.js
│       ├── no_chance.js
│       └── predict.js
└── Templates
    ├── chance.html
    ├── home.html
    ├── index.html
    └── no_chance.html

13 directories, 45 files
```

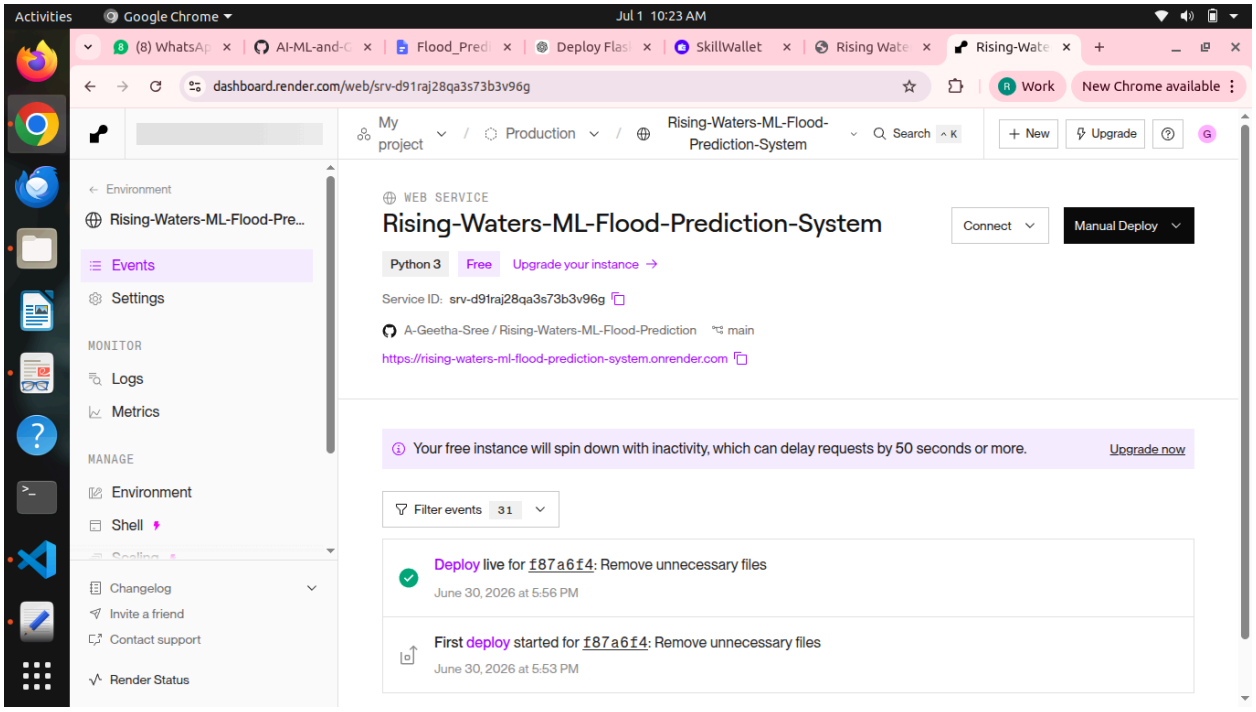

FLOOD-PREDICTION-SYS...
models
floods.save
transform.save
Notebook
.ipynb_checkpoints
Models
floods.save
transform.save
Flood_Predict... M
reports
Screenshots
Static
Templates
.gitignore
app.py
Dockerfile
Procfile
requirements_backu...
requirements.txt
app.py
26 def predict()
28 try:
81 else:
87 except Ex
88 retur
89
90
91 if __name__ =
PROBLEMS
OUTPUT
127.0.0.1 - - [01/J
127.0.0.1 - - [01/J
127.0.0.1 - - [01/J
127.0.0.1 - - [01/J
127.0.0.1 - - [01/J
127.0.0.1 - - [01/J
127.0.0.1 - - [01/J
127.0.0.1 - - [01/J
127.0.0.1 - - [01/J
127.0.0.1 - - [01/J



Git connection to Render



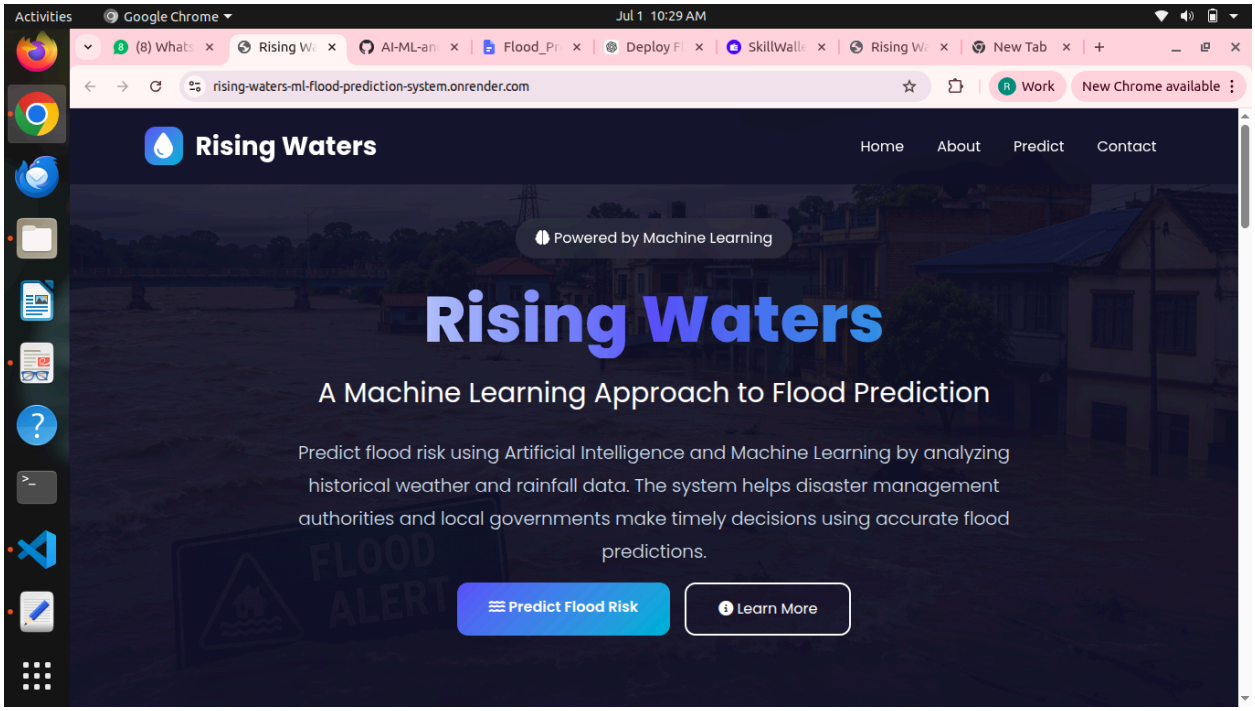
Render Dashboard

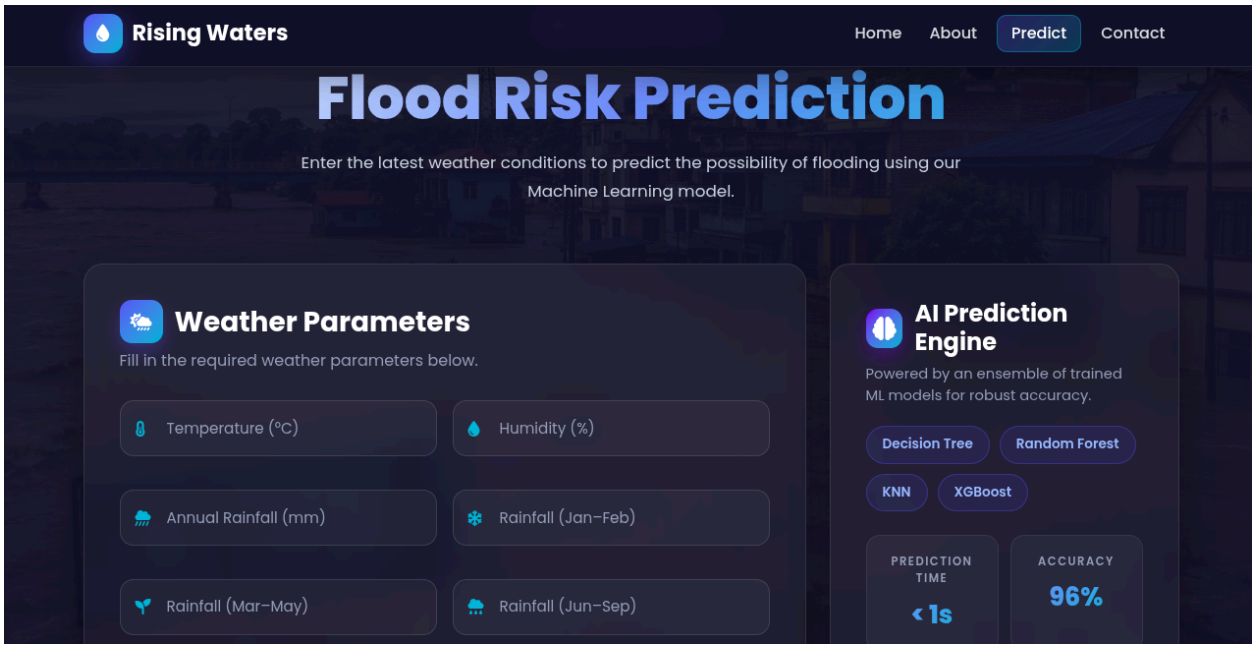


Usage Instructions:

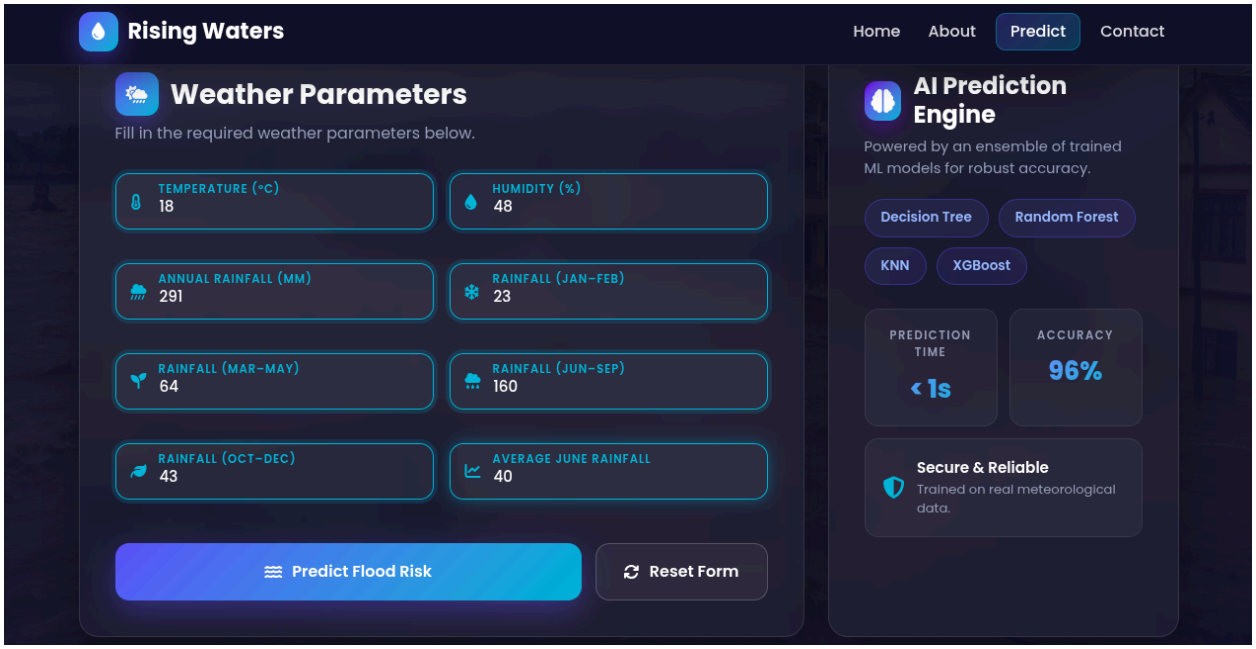
- Launch the application in a web browser.
- Open the Flood Prediction page.
- Enter the required weather and rainfall parameters.
- Click the Predict Flood Risk button.
- The application preprocesses the input data and sends it to the trained XGBoost model.
- View the prediction result, including:
- Modify the input values to test different weather conditions and generate new predictions.

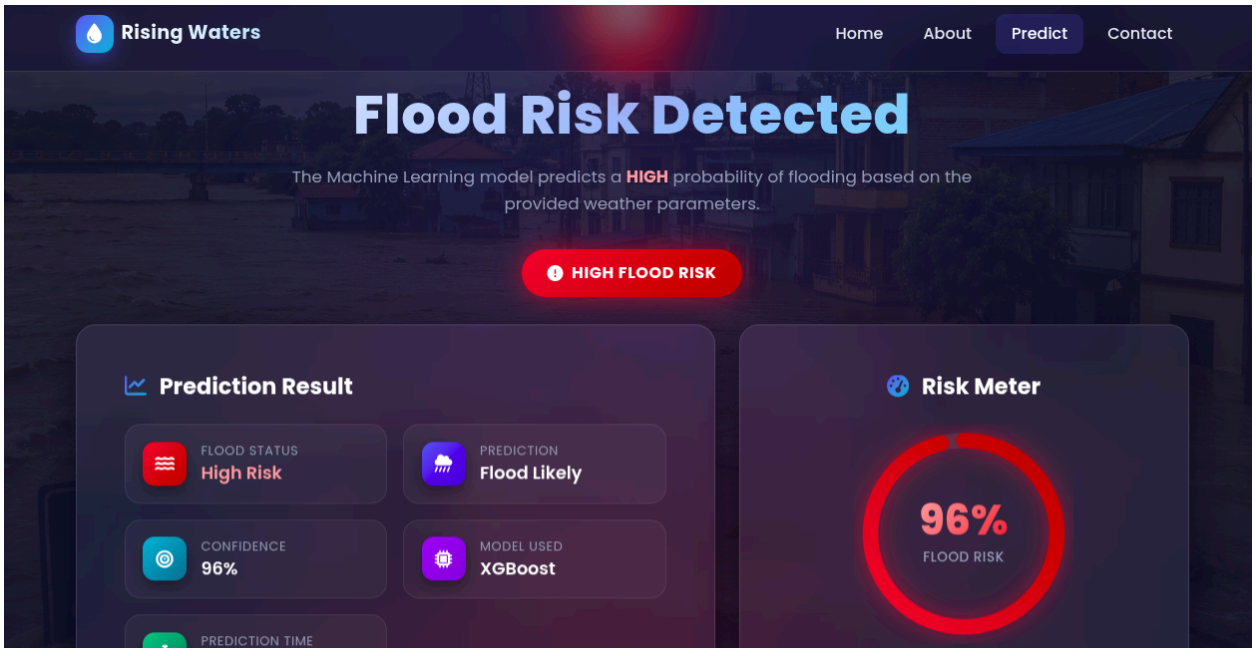
Live Website after Deployment



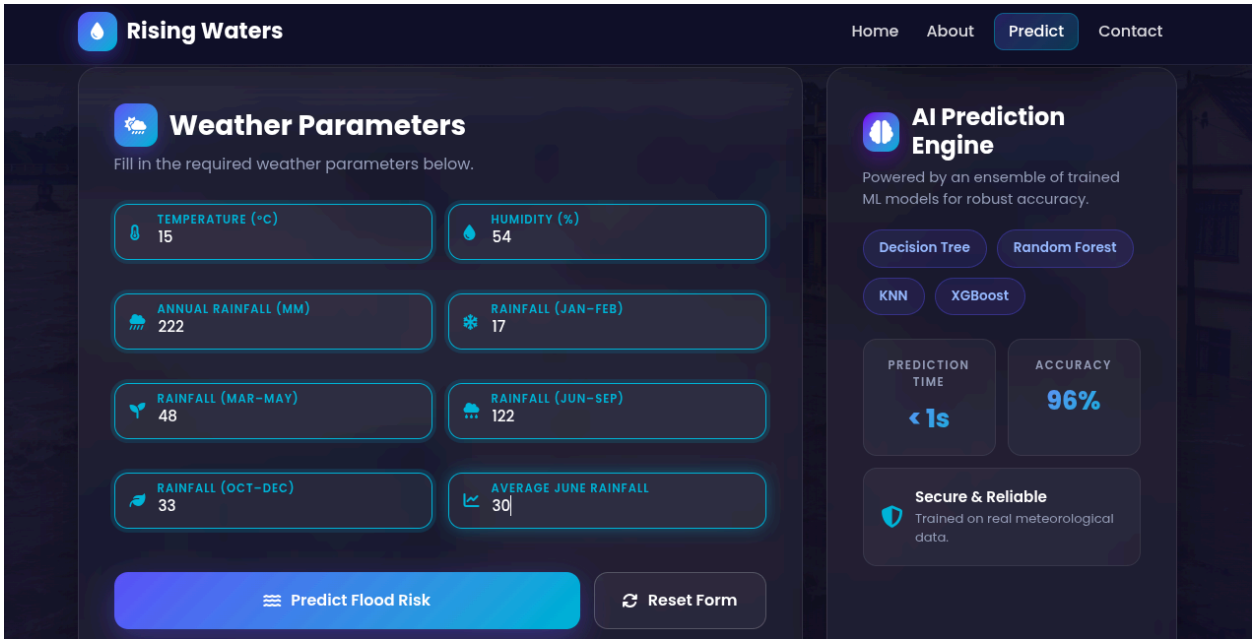


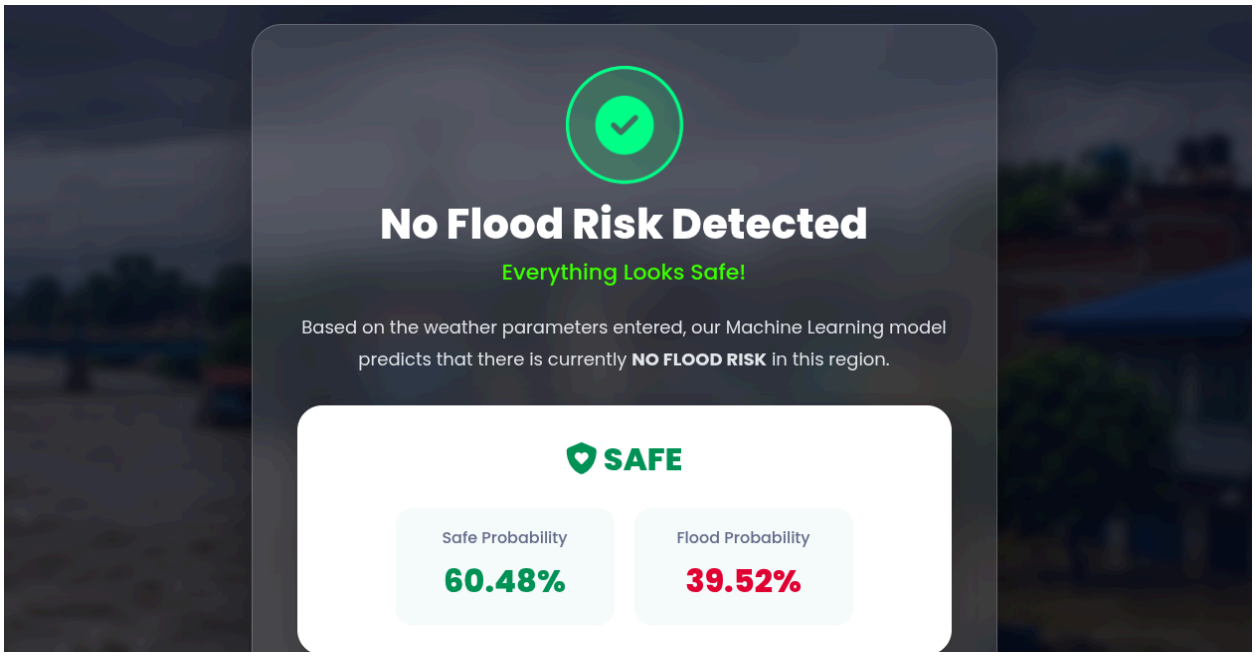
Flood


Rising Waters
Home
About
Predict
Contact
Weather Parameters
Fill in the required weather parameters below.
TEMPERATURE (°C)
18
HUMIDITY (%)
48
ANNUAL RAINFALL (MM)
291
RAINFALL (JAN-FEB)
23
RAINFALL (MAR-MAY)
64
RAINFALL (JUN-SEP)
160
RAINFALL (OCT-DEC)
43
AVERAGE JUNE RAINFALL
40
Predict Flood Risk
Reset Form
AI Prediction Engine
Powered by an ensemble of trained ML models for robust accuracy.
Decision Tree
Random Forest
KNN
XGBoost
PREDICTION TIME
< 1s
ACCURACY
96%
Secure & Reliable
Trained on real meteorological data.


Rising Waters
Home
About
Predict
Contact
Flood Risk Detected
The Machine Learning model predicts a HIGH probability of flooding based on the provided weather parameters.
HIGH FLOOD RISK
Prediction Result
FLOOD STATUS
High Risk
PREDICTION
Flood Likely
CONFIDENCE
96%
MODEL USED
XGBoost
PREDICTION TIME
Risk Meter
96%
FLOOD RISK

No Flood


Rising Waters
Home
About
Predict
Contact
Weather Parameters
Fill in the required weather parameters below.
TEMPERATURE (°C)
15
HUMIDITY (%)
54
ANNUAL RAINFALL (MM)
222
RAINFALL (JAN-FEB)
17
RAINFALL (MAR-MAY)
48
RAINFALL (JUN-SEP)
122
RAINFALL (OCT-DEC)
33
AVERAGE JUNE RAINFALL
30
Predict Flood Risk
Reset Form
AI Prediction Engine
Powered by an ensemble of trained ML models for robust accuracy.
Decision Tree
Random Forest
KNN
XGBoost
PREDICTION TIME
< 1s
ACCURACY
96%
Secure & Reliable
Trained on real meteorological data.


No Flood Risk Detected
Everything Looks Safe!
Based on the weather parameters entered, our Machine Learning model predicts that there is currently NO FLOOD RISK in this region.
SAFE
Safe Probability
60.48%
Flood Probability
39.52%

Team Member Details:

S.No	Team Member Name	Role	Responsibilities
1	Gayatri Botcha (Team Leader)	Built Machine Learning Model	Collected and analyzed the flood dataset, performed data preprocessing, trained and evaluated multiple machine learning models, selected the XGBoost model, and prepared the trained model for deployment.
2	A Geetha Sree (Team Member)	Website Building and Python Flask Backend Routing	Developed the Flask backend, designed the frontend using HTML, CSS, and JavaScript, integrated the trained machine learning model, performed application testing, managed the GitHub repository, and deployed the application on the Render cloud platform.

Important links:

Source code Link:
https://github.com/A-Geetha-Sree/Rising-Waters-ML-Flood-Prediction.git
Deployed Link:
https://rising-waters-ml-flood-prediction-system.onrender.com
Demo Link:
https://drive.google.com/file/d/1Oh6DDmfSZ0e0S9R2NFlMi3daJsoAN97Q/view?usp=drivesdk

Future Scope:

The Rising Waters – ML Flood Prediction System can be further enhanced by incorporating additional features and advanced technologies to improve prediction accuracy, usability, and scalability. Some potential future improvements include:

- Integrate real-time weather data using APIs from meteorological services.
- Add location-based flood prediction using GPS and GIS mapping.
- Use larger and more diverse datasets to improve model performance.
- Explore Deep Learning models such as LSTM or Neural Networks for enhanced prediction accuracy.
- Develop an interactive dashboard to visualize flood risk trends and historical data.
- Implement SMS, email, or mobile app notifications for early flood alerts.
- Store prediction history using a database such as MySQL or PostgreSQL.
- Support multi-language interfaces to improve accessibility for users from different regions.
- Deploy the application using containerization technologies such as Docker and Kubernetes for better scalability.
- Integrate the system with government disaster management platforms for real-time decision support.

Conclusion:

The Rising Waters – ML Flood Prediction System is a machine learning-based web application developed using Flask that helps predict flood risk based on weather and rainfall parameters. The system integrates a trained XGBoost machine learning model with a responsive frontend interface to provide users with quick and understandable flood predictions. The project includes dataset analysis, data preprocessing, model training, evaluation of multiple classification algorithms, backend development, frontend design, and cloud deployment using Render.

The application enables users to enter environmental parameters such as temperature, humidity, rainfall values, and seasonal rainfall data to determine the possibility of flood occurrence. The Flask backend processes the input, applies the saved feature transformation, and uses the trained model to generate prediction results with flood and safe condition probabilities.

The project demonstrates how machine learning and web technologies can be combined to develop an efficient decision-support system for flood risk analysis. The deployed application provides accessibility through an online platform and can be further enhanced in the future.

Future improvements may include integrating real-time weather APIs, using larger geographical datasets, adding location-based flood prediction, implementing deep learning models for improved accuracy, and developing an interactive dashboard for monitoring flood patterns. With continuous improvements in data quality and model performance, the system can evolve into a more advanced flood forecasting and early warning solution.

References

1. Python
   Pallets Projects. *Flask Web Framework Documentation.*
   Available at: https://flask.palletsprojects.com/
2. Scikit-learn Documentation
   Scikit-learn Developers. *Machine Learning in Python – Scikit-learn Documentation.*
   Available at: https://scikit-learn.org/stable/
3. XGBoost Documentation
   XGBoost Developers. *XGBoost Documentation – Scalable Machine Learning System.*
   Available at: https://xgboost.readthedocs.io/
4. Pandas Documentation
   The Pandas Development Team. *Pandas Python Data Analysis Library Documentation.*
   Available at: https://pandas.pydata.org/docs/
5. NumPy Documentation
   NumPy Developers. *NumPy User Guide and Reference Documentation.*
   Available at: https://numpy.org/doc/
6. Joblib Documentation
   Joblib Developers. *Joblib: Running Python Functions as Pipeline Jobs.*
   Available at: https://joblib.readthedocs.io/
7. Render Documentation
   Render. *Deploy and Host Applications on Render Cloud Platform.*
   Available at: https://render.com/docs
8. GitHub Documentation
   GitHub Docs. *Managing and Hosting Source Code Repositories.*
   Available at: https://docs.github.com/
9. HTML Documentation (MDN Web Docs)
   Mozilla Developer Network. *HTML, CSS, and JavaScript Documentation.*
   Available at: https://developer.mozilla.org/
10. OpenAI Machine Learning Resources (for general AI/ML concepts)
    OpenAI. *Artificial Intelligence and Machine Learning Resources.*
    Available at: https://openai.com/research/
11. World Meteorological Organization (WMO)
    World Meteorological Organization. *Weather, Climate, and Water Information Resources.*
    Available at: https://wmo.int/
12. Kaggle Datasets and Machine Learning Resources
    Kaggle. *Datasets and Machine Learning Community Resources.*
    Available at: https://www.kaggle.com/